MONLAU FORMACIÓ PROFESSIONAL	0612 Desenvolupament Web en Entorn Client
	RA1 · Práctica acumulativa de PHP para un jukebox
Alejandro Polo Diaz	
Práctica N. ª: 4	Practica Divertida

Resumen

Bloques 1 a 3. Arrancas PHP, creas una portada y describes una canción. Calculas su duración y decides qué mensaje mostrar.

Bloques 4 a 6. Construyes un catálogo con arrays, separas funciones y recibes datos mediante GET y POST. Aún no guardas cambios.

Bloques 7 y 8. Conservas la primera etapa, amplías Docker y lees el catálogo desde un JSON privado. Las bases de datos se prueban por separado; el CRUD seguirá usando JSON.

Bloques 9 y 10. Incorporas registro, login y sesión. Después añades creación, edición y eliminación de canciones sobre el mismo catálogo.

Bloques 11 y 12. Conectas el jukebox a los datos del servidor, realizas las pruebas finales y presentas el proyecto

Bloque 1 – Bienvenida (Hola Mundo)

Lo primero fue crear el archivo 01-bienvenida.php con un simple echo:

```php
<?php
echo 'Bienvenido al jukebox de la asociación';
```

Es solo una línea, pero sirve para verificar que el contenedor Docker con Apache y PHP está funcionando. Para levantarlo usé:

```
PS C:\Users\polod\Desktop\Alejandro\DAW 2\PHP\php-docker> docker compose up -d
```

Y para comprobar que respondía correctamente:

```
curl -i http://localhost:8090/01-bienvenida.php
```

Esto devolvió un HTTP 200 OK con el mensaje de bienvenida. También comprobé que no hubiera errores de sintaxis con php -l 01-bienvenida.php.

Bienvenido al jukebox de la asociación

Bloque 2 – Portada y tipos de datos

Aquí creé la página principal (02-portada.php). Aprendí a mezclar PHP con HTML y a usar htmlspecialchars() para escapar los datos antes de mostrarlos, que es muy importante para evitar ataques XSS:

```php
<?php
$saludo = 'El catálogo musical está en preparación.';
?>
<h1>Jukebox de la asociación</h1>
<p><?= htmlspecialchars($saludo, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8') ?></p>
```

Lo de htmlspecialchars() al principio me pareció un rollo porque es muy largo de escribir, pero entiendo que es necesario para que si alguien mete código malicioso en una variable, no se ejecute en el navegador.

Jukebox de la asociación

El catálogo musical está en preparación.

Ver el saludo inicial

Sesión de prueba

La canción Sesión de prueba es de Grupo del aula.

Duración de prueba: 185 segundos.

Bloque 3 – Estructuras de control (duración)

En este bloque practiqué las estructuras condicionales (if/elseif/else). La idea es clasificar una canción según su duración:

```php
<?php
$duration = 185;

if ($duration === 0) {
    $status = 'Sin duración';
} elseif ($duration <= 240) {
    $status = 'Hasta cuatro minutos';
} else {
    $status = 'Más de cuatro minutos';
}
```

También aprendí a convertir segundos a minutos con intdiv() (división entera) y el operador módulo %:

```php
$minutes = intdiv($duration, 60);   // 3
$seconds = $duration % 60;          // 5
```

3 minutos y 5 segundos.

Hasta cuatro minutos

Bloque 4 – Arrays y catálogo

Aquí empecé a usar arrays para guardar las canciones. Practiqué con for, while y foreach. El que más uso es foreach porque es el más cómodo para recorrer listas:

```php
<?php
$titles = ['Sesión de prueba', 'Ritmo nocturno', 'Tema de película'];
?>
<ul>
    <?php foreach ($titles as $title) { ?>
        <li><?= htmlspecialchars($title, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8') ?></li>
    <?php } ?>
</ul>
```

También probé el caso de un catálogo vacío para que mostrara un mensaje en vez de una lista en blanco.

Contador con for

1. 1
2. 2
3. 3

Contador con while

1. 1
2. 2
3. 3

Títulos

- Sesión de prueba
- Ritmo nocturno
- Tema de película

Catálogo de canciones

- **Sesión de prueba** — Grupo del aula, 3 minutos y 5 segundos
- **Ritmo nocturno** — Artista de ejemplo, 4 minutos y 0 segundos
- **Tema de película** — Compositor de prueba, 3 minutos y 35 segundos

Bloque 5 – Funciones

Definí funciones propias en un archivo aparte (lib/canciones.php) para poder reutilizarlas. Por ejemplo, una función que convierte segundos a texto legible:

```php
function duracionLegible(int $total): string
{
    $minutes = intdiv($total, 60);
    $seconds = $total % 60;
    return $minutes . ' minutos y ' . $seconds . ' segundos';
}
```

Y una función escapar() que simplifica la llamada a htmlspecialchars():

```php
function escapar(mixed $value): string
{
    return htmlspecialchars((string) $value, ENT_QUOTES | ENT_SUBSTITUTE, 'UTF-8');
}
```

Para usar estas funciones en otros archivos utilicé require_once. Aprendí que require lanza un error fatal si el archivo no existe, mientras que include solo da un warning.

60 segundos: 1 minutos y 0 segundos

185 segundos: 3 minutos y 5 segundos

Bloque 6 – Formularios y filtros

Implementé un filtro de canciones por categoría usando $_GET. El código valida que la categoría sea una de las permitidas; si no lo es, devuelve un error 400:

```php
$category = $_GET['category'] ?? 'todas';

if (!in_array($category, $allowedCategories, true)) {
    http_response_code(400);
    $invalidCategory = true;
} else {
    $songs = array_values(array_filter(
        $songs,
        fn (array $song): bool => $song['category'] === $category
    ));
}
```

Me gustó la parte de array_filter() con funciones flecha (fn =>), que es una forma bastante limpia de filtrar arrays.

Catálogo por categoría

- Ritmo nocturno — Artista de ejemplo

Catálogo por categoría

Categoría no válida.



Bloque 7 – Docker multiservicio (Etapa 2)

Este fue el bloque más complejo. Creé un compose.yaml con tres servicios: Apache+PHP, PostgreSQL y MariaDB:

```yaml
services:
  web:
    build:
      context: ..
      dockerfile: docker/Dockerfile
    ports:
      - "127.0.0.1:8091:80"
    volumes:
      - ./src:/var/www/html
      - ./data:/var/www/data
    depends_on:
      postgres:
        condition: service_healthy
      mariadb:
        condition: service_healthy

  postgres:
    image: postgres:17-bookworm
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U \"${POSTGRES_USER}\""]

  mariadb:
    image: mariadb:11.4
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
```

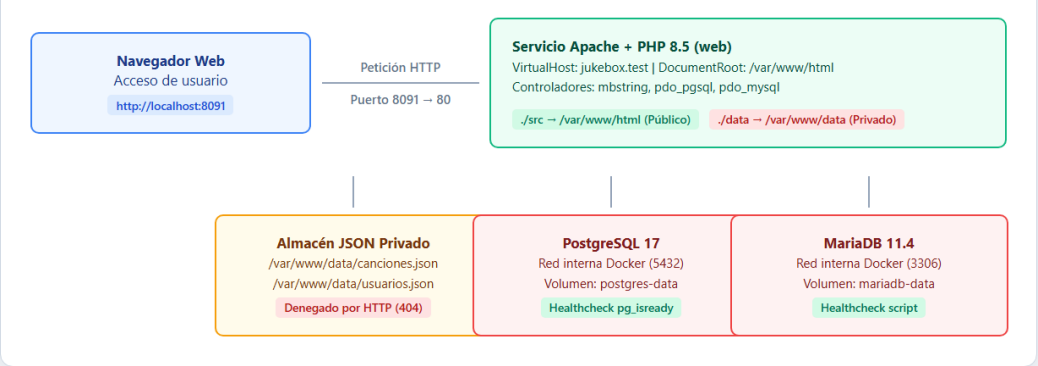
Para levantarlo usé:

```
PS C:\Users\polod\Desktop\Alejandro\DAW 2\PHP\php-docker> cd docker
PS C:\Users\polod\Desktop\Alejandro\DAW 2\PHP\php-docker> docker compose up -d
```

Después verifiqué las conexiones a ambas bases de datos con 11-conexiones.php, que usa PDO:

```php
$connection = new PDO(
    'pgsql:host=' . getenv('PGHOST') . ';dbname=' . getenv('PGDATABASE'),
    getenv('PGUSER'),
    getenv('PGPASSWORD')
);
```

El comando curl http://localhost:8091/11-conexiones.php devolvió un JSON con "status": "ok" para las dos bases de datos.



Bloque 8 – Persistencia JSON

Los datos del catálogo se guardan ahora en /var/www/data/canciones.json, que está fuera del directorio público. Así nadie puede acceder al JSON directamente por URL (devuelve 404).

El archivo lib/almacen-json.php se encarga de leer y escribir el JSON con bloqueo de archivos (flock) para evitar problemas si dos usuarios escriben a la vez:

```php
const ARCHIVO_CANCIONES = '/var/www/data/canciones.json';

function leerCancionesJson(): array
{
    return leerListaJson(ARCHIVO_CANCIONES, 'songs');
}
```

Catálogo JSON

- **Sesión de prueba** — Grupo del aula, 3 minutos y 5 segundos
- **Ritmo nocturno** — Artista de ejemplo, 4 minutos y 0 segundos
- **Tema de película** — Compositor de prueba, 3 minutos y 35 segundos



Bloque 9 – Autenticación (registro y login)

Implementé registro y login con sesiones PHP. Las contraseñas se guardan hasheadas con password_hash(), que es la forma segura que recomienda PHP:

```
$hash = password_hash($password, PASSWORD_DEFAULT);
```

Para proteger los formularios contra ataques CSRF, se genera un token aleatorio que se incluye como campo oculto:

```
<input type="hidden" name="csrf" value="<?= escapar(tokenCsrf()) ?>">
```

Si alguien intenta enviar el formulario sin el token correcto, el servidor rechaza la petición con un error 403 Forbidden.

La validación del registro comprueba varias cosas: que el alias tenga entre 2 y 30 caracteres, que el email sea válido, que la contraseña tenga al menos 12 caracteres, y que la confirmación coincida.

Crear cuenta

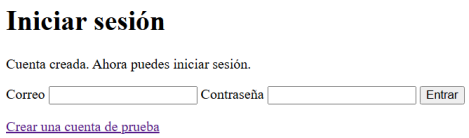
Alias Correo Contraseña (12 a 64 caracteres) Repetir contraseña
Crear cuenta

Ya tengo una cuenta

Crear cuenta

Alias AlejandroDAW Correo alejandro@asociacion.org Contraseña (12 a 64 caracteres) Repetir contraseña Las contraseñas no coinciden. Crear cuenta

Ya tengo una cuenta



Bloque 10 – CRUD de canciones

Con el usuario ya autenticado, se pueden hacer las cuatro operaciones básicas: Crear, Leer, Actualizar y Eliminar canciones. La lista principal (canciones.php) lee los datos del JSON y los muestra con un foreach:

```php
<?php foreach ($songs as $song) { ?>
    <li>
        <strong><?= escapar($song['title']) ?></strong>
        – <?= escapar($song['artist']) ?>
        <a href="cancion-editar.php?id=<?= escapar($song['id']) ?>">Editar</a>
        <a href="cancion-eliminar.php?id=<?= escapar($song['id']) ?>">Eliminar</a>
    </li>
<?php } ?>
```

Para generar IDs nuevos se usa max(id) + 1 y al eliminar se reindexa con array_values().

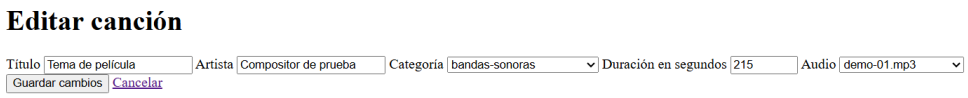
Catálogo de canciones

Inicio Añadir canción

- **Sesión de prueba** — Grupo del aula · electronica · 3 minutos y 5 segundos Editar Eliminar
- **Ritmo nocturno** — Artista de ejemplo · rock · 4 minutos y 0 segundos Editar Eliminar
- **Tema de película** — Compositor de prueba · bandas-sonoras · 3 minutos y 35 segundos Editar Eliminar

Añadir canción

Título Sinfonía Electrónica Artista DJ DAW Categoría electronica Duración en segundos 245 Audio demo-01.mp3
Guardar canción Cancelar





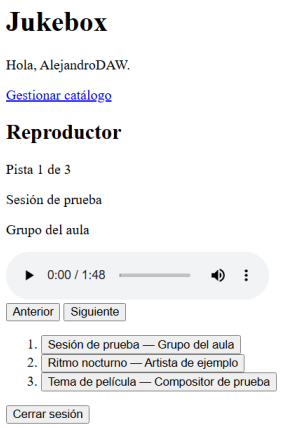
Bloque 11 – Home y API REST

Se creó la página de inicio del Jukebox y una API REST en api/canciones.php. La API devuelve las canciones en JSON, pero solo si el usuario tiene sesión iniciada:

```php
if (!isset($_SESSION['usuario_id'])) {
    http_response_code(401);
    echo json_encode(['error' => 'Autenticación requerida.']);
    exit;
}

$songs = leerCancionesJson();
echo json_encode($songs);
```

Para probar la API usé curl http://localhost:8091/api/canciones.php. Sin sesión devuelve 401, y con la cookie de sesión devuelve el listado en JSON.



Bloque 12 – Matriz de aceptación

Para finalizar, hice una matriz de aceptación donde se comprueba que cada requisito de la práctica está cubierto. Todos los puntos están marcados como completados.

Prueba	Preparación	Resultado Esperado	Resultado Real	Estado
Etapa inicial	docker compose up -d en raíz	PHP responde en 8090	HTTP 200 OK con saludo oficial	✓ SUPERADA
Etapa ampliada	docker compose up -d --build en docker/	Web 8091 y 2 DBs saludables	3 contenedores activos, DBs healthy	✓ SUPERADA
Conexión PDO	GET /11-conexiones.php	Dos estados ok, sin contraseñas	JSON ok para PostgreSQL y MariaDB (200)	✓ SUPERADA
Registro usuario	POST /register.php con CSRF	Alta única en JSON con hash	Usuario guardado en JSON, redirect 303	✓ SUPERADA
Registro inválido	POST con claves distintas o email dup.	Rechazo sin escribir datos	HTTP 400 y mensajes de error específicos	✓ SUPERADA
Aislamiento datos	GET /data/usuarios.json	Denegar descarga de datos privados	HTTP 404 Not Found (Apache aísla data/)	✓ SUPERADA
Control acceso	GET /home.php y /canciones.php	Redirección a login si no hay sesión	HTTP 303 Location: login.php	✓ SUPERADA
Protección CSRF	POST sin token o token alterado	HTTP 403 Forbidden sin mutación	HTTP 403 "Solicitud no válida."	✓ SUPERADA
CRUD Canciones	Create, Read, Update, Delete por id	Solo muta la ficha indicada	Alta id=4, Edita id=3, Borra id=2 reindexado	✓ SUPERADA
Método / ID	GET mutador, id inexistente o texto	Sin cambios; códigos 405/404/400	GET a logout da 405; id=999 da 404; id=abc da 400	✓ SUPERADA
Persistencia	docker restart web (conserva data/)	Catálogo y cuentas persisten	canciones.json y usuarios.json intactos	✓ SUPERADA
API Canciones	GET /api/canciones.php	401 JSON sin sesión; 200 con sesión	Sin sesión: 401 JSON; Con sesión: array plano	✓ SUPERADA

Conclusiones

Esta práctica me ha servido para aprender PHP de forma progresiva. He ido desde un simple echo hasta montar una aplicación completa con Docker, bases de datos, autenticación y API. Lo que más me costó fue la parte de Docker con varios servicios y entender bien la protección CSRF, pero al final cuando ves cómo encajan todas las piezas tiene bastante sentido. En general creo que es una buena práctica para ir cogiendo soltura con el desarrollo web del lado del servidor.